SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING **D. Harsha Vardhan Chowdary**
COURSE CODE: CSA4303 **192372030**

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: **Total Marks: 20**

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: **DOM Structure Analysis**

Consider the following DOM tree representation:

```
<html>
  <head>
  <title>TestPage</title>
</head>
  <body>
  <div>
  <p>HelloWorld
  </div>
 </body>
 </html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

I Dabbara. Harsha Vardhan Chowdary (192372030) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Question 1: HTTP Response Analysis

1. Identify Incorrect or Invalid Fields

- Content-Length: -150: The Content-Length header value is invalid. Under HTTP/1.1 specifications (RFC 9110), Content-Length must be a non-negative decimal integer representing the exact octet count (byte size) of the message body. A negative value (-150) violates protocol syntax.

2. Analysis of Content-Length Issue and Its Impact

- **Parsing Failure:** HTTP clients (browsers) rely on Content-Length to determine how many bytes to read from the socket before concluding the response stream.
- **Connection Drops or Timeouts:** Receiving a negative byte count causes parser exceptions. The browser cannot determine payload boundaries, leading to premature stream termination, stalled connection loops, or socket read errors.
- **HTTP Request Smuggling Vulnerability:** Misformed content length headers can desynchronize front-end reverse proxies and backend application servers, creating security risks such as HTTP response splitting or request smuggling.

3. Status Code Appropriateness

- **Status Code:** 200 OK indicates a successful HTTP transaction.
- **Mismatch:** Returning 200 OK alongside a malformed protocol header is contradictory. While the application logic on the server may have executed without errors, the HTTP response envelope itself is structurally invalid.

4. Corrected Header Configuration

Assuming a body payload of 150 bytes:

```
HTTP
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 150
Connection: keep-alive
```

Question 2: DOM Structure Analysis

1. Missing or Improperly Closed Elements

- **Unclosed** **Element:** The paragraph element is opened inside but is never explicitly closed ( ).
- **Block Element Nesting:** Structural hierarchy ends abruptly at the boundary without terminating child paragraph content.

2. Browser Interpretation & DOM Construction

- HTML5 parsing algorithms enforce automatic tag closure when encountering explicit closing tags of parent elements.
- When the browser parser reaches , it detects an open tag inside it. To construct a valid DOM node tree, the browser automatically injects an implicit closing tag right before the .

- **Constructed DOM Tree:**

HTML
html
├── head
│ └── title ("TestPage")
└── body
└── div
└── p ("HelloWorld")

3. Impact on Rendering and Layout

- **Layout Shifts:** Modern engines (Blink, Gecko) recover smoothly from unclosed <p> tags when followed directly by . However, nested inline elements or trailing CSS rules applied to p vs div can cascade unexpectedly.
- **Inherited CSS Styling Errors:** Rules targeting div > p or :last-child selectors may apply incorrectly if implicit closing tags alter intended node positions in complex layouts.

4. Corrected Valid Markup

HTML
```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <title>TestPage</title>
</head>
<body>
 <div>
  <p>HelloWorld</p>
 </div>
</body>
</html>
```

Question 3: GET vs POST in Login System

1. Data Transmission Differences

Attribute	GET Method	POST Method
Data Location	Appended to URL as query string parameters (/login?user=admin)	Enclosed inside the HTTP request payload body
Data Visibility	Fully visible in browser address bar, history, and network logs	Obscured from browser address bar and referrer headers
Data Size Limit	Restricted by URL length limits (~2048 chars depending on browser)	Unrestricted (limited only by server configuration)
Caching & History	Cached by default; remains stored in browser history	Not cached; parameters are not preserved in history

2. Suitable Method for Authentication

- **Choice:** POST is the only acceptable standard method for submitting login credentials.
- **Reasoning:** POST isolates sensitive information (usernames, passwords) inside the request message body rather than exposing it in the address bar, browser logs, or external referrers.

3. Security Risks of Using GET for Authentication

1. **URL Exposure & Caching:** Passwords remain stored in plain text within browser browsing history, server access logs (e.g., Apache/Nginx access logs), and web proxy caches.
2. **Referrer Header Leaks:** Navigating to external links from a GET-authenticated page leaks credentials via the HTTP Referer header to external third-party servers.
3. **Shoulder Surfing:** Clear-text display of passwords in the browser address bar makes credentials vulnerable to shoulder surfing.

4. Best Practices for Secure Login Implementation

- **HTTPS / TLS Encryption:** Serve all authentication routes strictly over HTTPS (https://) to protect payload data in transit from wiretapping and Man-in-the-Middle (MitM) attacks.
- **HTTP POST Submission:** Transmit form data using method="POST".
- **CSRF Protection:** Include anti-CSRF tokens in login forms to ensure requests originate from legitimate users.
- **Security Headers:** Set Cache-Control: no-store and Pragma: no-cache response headers on login routes to prevent caching sensitive responses.

Question 4: HTML Code Inspection:

1. Missing Closing Tags

- **Unclosed** <h1>**:** The <h1> tag opens before Welcome and is never closed with </h1>.
- **Unclosed** **:** The tag opens before This is a paragraph and lacks .

2. Issues with the <img> **Tag**

- **Missing** alt **Attribute:** Violates accessibility standards (WCAG 2.1). Screen readers cannot describe the image to visually impaired users.
- **Self-Closing Syntax in XHTML Contexts:** While void elements do not require closing tags in HTML5, omitting the closing slash (/>) or self-containment breaks strict XHTML compliance.
- **Missing Dimensional Attributes:** Lacks width and height attributes, leading to Cumulative Layout Shift (CLS) during image load.

3. Browser Error Recovery Behavior

- Browsers implicitly auto-close <h1>Welcome when encountering the block-level tag, separating them into sequential sibling elements.
- Browsers auto-close when encountering </body>.
- **Parsed Element Result:**

HTML

```
<h1>Welcome</h1>
<p>This is a paragraph</p>
<img src="image.jpg">
```

4. Corrected W3C-Compliant Code

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Sample</title>
</head>
<body>
 <h1>Welcome</h1>
 <p>This is a paragraph</p>
 <img src="image.jpg" alt="Sample descriptive image text" width="600" height="400">
</body>
</html>
```

Question 5: Browser-Server Communication Flow

1. Interpretation of Flow Steps

1. **URL Entry:** The user inputs a URL (e.g., http://example.com/index.html) into the browser address bar.
2. **DNS Lookup:** The browser resolves the human-readable domain name (example.com) to its corresponding numerical IPv4/IPv6 IP address (93.184.216.34).
3. **HTTP Request:** The browser establishes a TCP connection (via a 3-way handshake) and sends an HTTP GET or POST request header/body payload to the server IP.
4. **Server Processing:** The web server (e.g., Apache, Nginx, Node.js) accepts the request, queries database layers if needed, and executes server-side scripts.
5. **Response Generation:** The server returns an HTTP response containing headers (status codes like 200 OK) and the requested content payload (HTML, CSS, JSON).
6. **Browser Rendering:** The browser parses HTML to construct the DOM tree, parses CSS for the CSSOM tree, combines them into a Render Tree, and paints the layout onto the screen.

2. Latency Bottlenecks

[1. User Entry] ➔ [2. DNS Lookup] ➔ [TCP / TLS Handshake] ➔ [3. HTTP Request] ➔ [4. Server Execution] ➔ [5. Response Stream] ➔ [6. DOM Parsing & Render]

| | | | |

Cold DNS Lookup Network RTT Database Queries Unoptimized Assets

- **Step 2 (DNS Lookup):** Uncached DNS queries requiring recursive resolver lookups across root and TLD servers add high latency.
- **Network Round-Trip Time (RTT):** Establishing initial TCP handshakes and TLS security negotiation adds multiple network round trips.
- **Step 4 (Server Execution):** Unoptimized database queries, heavy synchronous backend operations, or legacy server logic delay response delivery (High Time to First Byte - TTFB).

- **Step 6 (Rendering):** Large uncompressed assets, render-blocking CSS/JS, and excessive DOM node depth cause visual lag and execution bottlenecks.

3. Role of DNS

- **Translation Bridge:** Translates human-friendly domain names into machine-routable IP addresses required for IP network routing.
- **Caching Hierarchy:** Operates across multiple caching layers (Browser Cache → OS Cache → Router Cache → ISP Resolver Cache) to minimize resolution times on repeated web visits.

4. Performance Optimizations

- **DNS Optimization:** Implement DNS prefetching (<link rel="dns-prefetch" href="//example.com">) and leverage fast global DNS providers (Cloudflare, Google Public DNS).
- **Content Delivery Networks (CDNs):** Cache static assets at edge servers physically closer to end users to reduce Network RTT.
- **Browser Caching & HTTP/2 or HTTP/3:** Use Cache-Control headers for static files and utilize multiplexed HTTP/2 or QUIC (HTTP/3) transport layers.
- **Asset Optimization:** Minify HTML/CSS/JavaScript, enable Gzip/Brotli response compression, and optimize images into modern formats (WebP/AVIF).